

Faculty of Engineering

Mechanical Engineering · Spring 2025/2026

Milestone 3 Report

Signals and Image Processing ENMR602

Custom Convolutional Neural Network

Supervised by:

Dr. Moheb Mekhaiel

Eng. Moaz Fouda

Eng. Hisham Hafez

Eng. Yasmine Attia

Team Members

Name	ID	Tutorial	Major
Marwan Hesham Elqady	13006158	10	Robotics and Automation
Moaaz Bassouny	13001998	10	Robotics and Automation
Ahmed Mohamed Ramadan	13005623	10	Robotics and Automation
Youssef Mokhtar	13005346	10	Robotics and Automation

German International University

Spring 2025/2026

Contents

1	Introduction	2
2	Dataset	2
3	Milestone 3 Custom CNN Pipeline	2
4	Custom CNN V4 Architecture	3
4.1	Residual-Squeeze-Excitation Block	3
5	Training Configuration	4
6	Milestone 3 Evaluation Results	4
7	Confusion Matrix	5
8	Top 3 Probabilities	6
9	Milestone 2 Updated Pipeline	7
10	Milestone 2 Feature Vector	8
11	Milestone 2 Confusion Matrix and Runtime	8
12	Comparison Between Milestone 2 and Milestone 3	9
13	Discussion	9
14	Conclusion	9
A	Code Availability	10
B	Core Architecture Code Excerpt	10

1 Introduction

This report presents the final Milestone 3 solution for classifying six ball classes using a custom Convolutional Neural Network (CNN). The target classes are american football, baseball, basketball, football, golf ball, and tennis ball. The dataset was used exactly as provided, with the original training and testing folders preserved.

A rule clarification stated that predefined deep learning models are not allowed. Therefore, the final solution does not use MobileNet, EfficientNet, DenseNet, ResNet, ImageNet weights, transfer learning, or any architecture from `tf.keras.applications`. The final model was manually configured using custom CNN layers such as convolution, batch normalization, pooling, dropout, dense layers, residual connections, and squeeze-excitation style attention modules.

The final custom model, named **Custom CNN V4 SE HighRes**, achieved a test accuracy of **97.58%**. This result improves significantly over the Milestone 2 classical machine learning solution, which achieved **89.97%** using Random Forest + LBP.

2 Dataset

The dataset contains RGB images divided into a training folder and a testing folder. The original train/test split was preserved. No images were added, removed, replaced, edited, or moved between folders. During loading, images were resized in memory to a fixed model input size of $256 \times 256 \times 3$.

Table 1: Dataset distribution per class.

Class	Train	Test	Total
american_football	271	50	321
baseball	237	49	286
basketball	224	50	274
football	212	45	257
golf_ball	212	45	257
tennis_ball	251	50	301
Total	1407	289	1696

The training folder was split into training and validation subsets using a 15% validation split. The testing folder was kept unchanged and used only for final evaluation.

3 Milestone 3 Custom CNN Pipeline

The final Milestone 3 pipeline is:

Read Image $\rightarrow$ Resize $\rightarrow$ Augment During Training $\rightarrow$ Custom CNN Feature Extraction $\rightarrow$ Dense Class

The preprocessing and training steps were:

1. Read images from the original train and test folders.
2. Resize images in memory to $256 \times 256 \times 3$.
3. Apply training-only augmentation: horizontal flip, small rotation, zoom, and contrast variation.
4. Rescale pixel values to the range $[0, 1]$.
5. Extract features using manually built convolutional and residual squeeze-excitation blocks.
6. Classify using fully connected layers and a six-neuron softmax output layer.

The augmentation was applied only during training and did not modify the stored dataset files.

4 Custom CNN V4 Architecture

The final model was manually configured from scratch. It uses convolutional blocks, residual skip connections, and manually implemented squeeze-excitation style channel attention. No predefined model architecture or pretrained weights were used.

Table 2: Final Custom CNN V4 architecture summary.

Stage	Layers / Operation	Purpose
Input	$256 \times 256 \times 3$ RGB image	Fixed CNN input size
Augmentation	RandomFlip, RandomRotation, RandomZoom, RandomContrast	Improve generalization
Rescaling	Pixel values divided by 255	Normalize image intensity
Stem	Conv2D(32) + BN + Swish, repeated twice, then MaxPooling	Initial low-level feature extraction
Stage 1	Two residual-SE blocks with 64 filters	Shallow texture and edge patterns
Stage 2	Two residual-SE blocks with 128 filters	Medium-level shape and colour patterns
Stage 3	Two residual-SE blocks with 192 filters	Larger object structures
Stage 4	Two residual-SE blocks with 256 filters	High-level discriminative features
Stage 5	One residual-SE block with 384 filters	Final deep feature extraction
Head	GlobalAveragePooling, BatchNorm, Dropout, Dense(512), Dense(256)	Classification feature reduction
Output	Dense(6) + Softmax	Six-class probability output
Total parameters	6,927,030	–

4.1 Residual-Squeeze-Excitation Block

Each residual-SE block contains two convolutional layers. A shortcut connection is added to improve gradient flow during training. The squeeze-excitation part uses global average pooling and dense layers to learn channel-wise feature importance. This attention mechanism is manually implemented using standard Keras layers.

5 Training Configuration

Table 3: Training configuration for the final custom CNN.

Item	Value
Framework	TensorFlow/Keras
Model type	Custom CNN from scratch
Input size	$256 \times 256 \times 3$
Number of classes	6
Loss function	Categorical cross-entropy with label smoothing = 0.02
Optimizer	AdamW, learning rate = 5×10^{-4}
Weight decay	8×10^{-5}
Validation split	15% of training folder
Callbacks	ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
Training runtime	1535.37 seconds / 25.59 minutes
Total parameters	6,927,030

The best model was saved according to validation accuracy, and early stopping was used to avoid unnecessary training after validation performance stopped improving.

6 Milestone 3 Evaluation Results

The final model was evaluated on the unchanged test set of 289 images. The overall results are shown in Table 4.

Table 4: Final Milestone 3 Custom CNN V4 evaluation metrics.

Metric	Value
Test accuracy	97.58%
Macro precision	0.9771
Macro recall	0.9750
Macro F1-score	0.9757
Weighted precision	0.9766
Weighted recall	0.9758
Weighted F1-score	0.9759

Table 5: Classification report for the final Custom CNN V4 model.

Class	Precision	Recall	F1-score	Support
american_football	0.9245	0.9800	0.9515	50
baseball	1.0000	0.9592	0.9792	49
basketball	0.9804	1.0000	0.9901	50
football	1.0000	0.9556	0.9773	45
golf_ball	0.9773	0.9556	0.9663	45
tennis_ball	0.9804	1.0000	0.9901	50
Macro average	0.9771	0.9750	0.9757	289
Weighted average	0.9766	0.9758	0.9759	289

7 Confusion Matrix

The final confusion matrix is shown in Table 6. The model correctly classified 282 out of 289 test images, giving 7 total errors.

Table 6: Final Custom CNN V4 confusion matrix.

True / Predicted	american_football	baseball	basketball	football	golf_ball	tennis_ball
american_football	49	0	1	0	0	0
baseball	2	47	0	0	0	0
basketball	0	0	50	0	0	0
football	1	0	0	43	1	0
golf_ball	1	0	0	0	43	1
tennis_ball	0	0	0	0	0	50

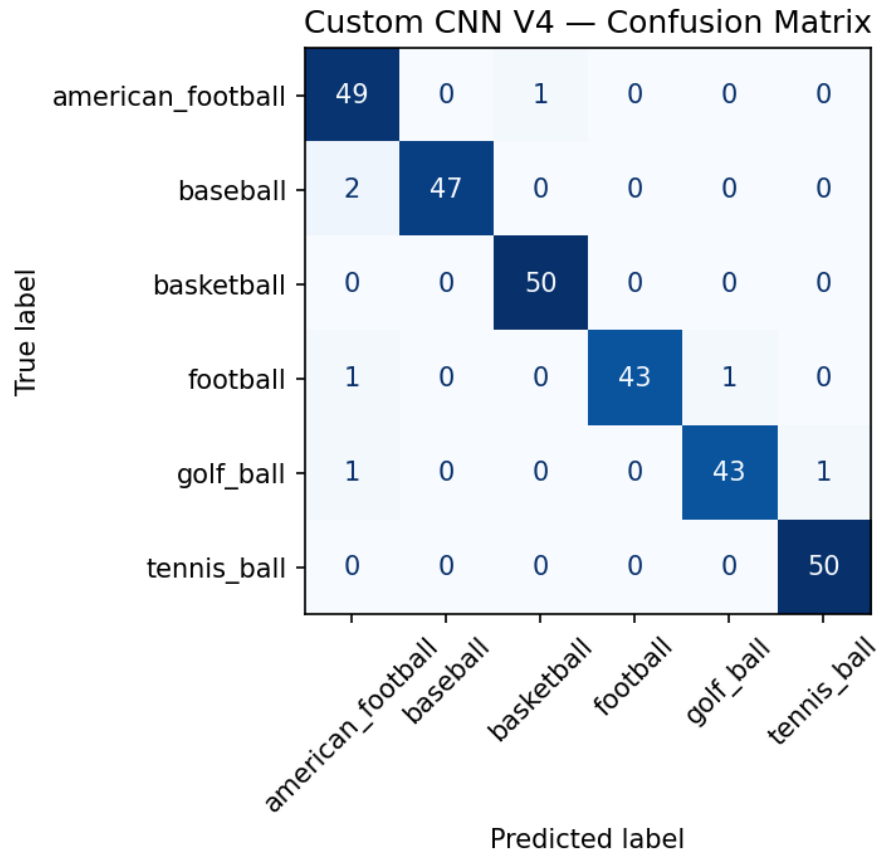


Figure 1: Final Custom CNN V4 confusion matrix.

The strongest classes were basketball and tennis ball, each reaching 100% recall. Most remaining errors occurred between visually similar classes or images with unusual viewpoints.

8 Top 3 Probabilities

Figure 2 shows selected test images with their true label, predicted label, and the top three predicted probabilities from the softmax output layer.

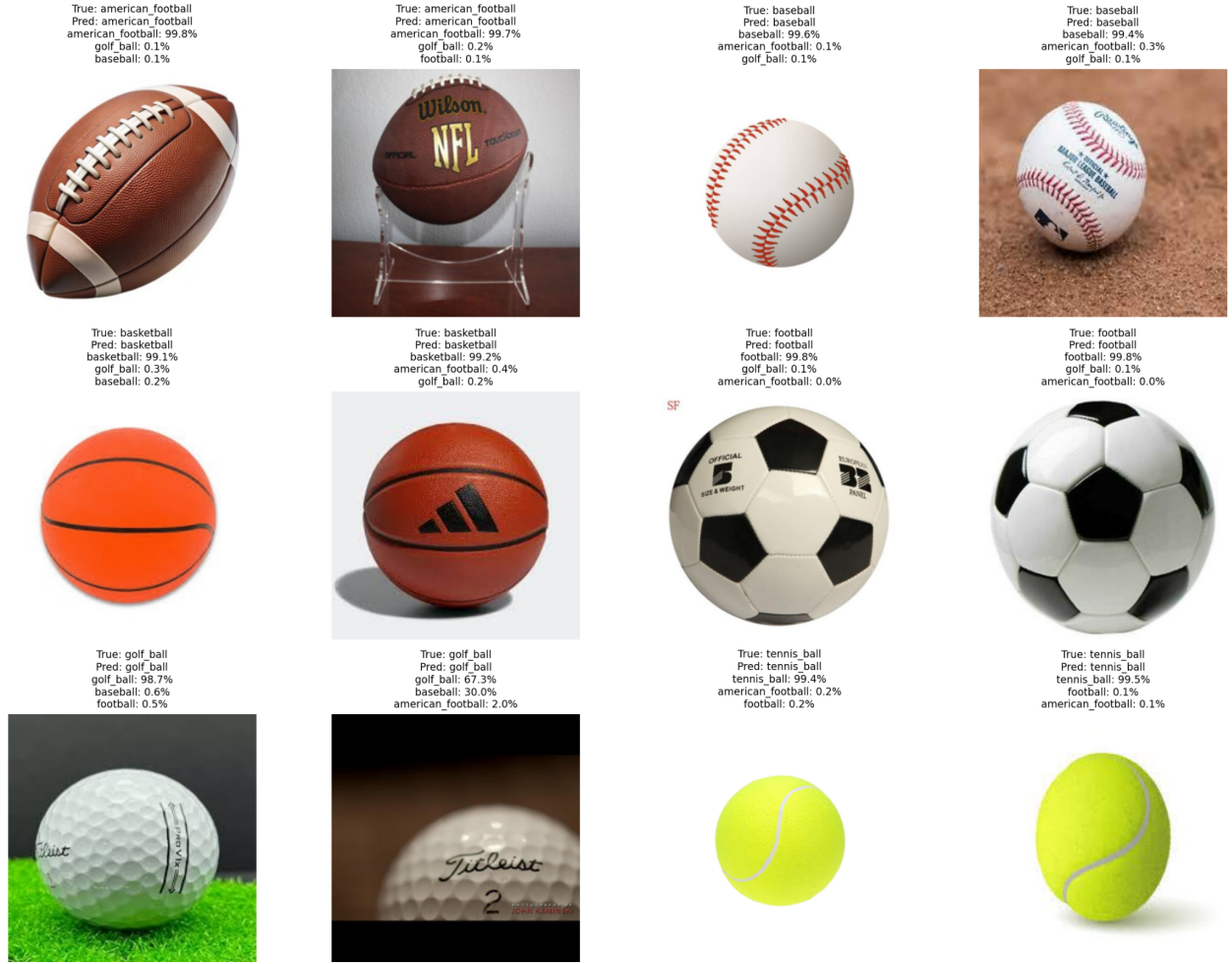


Figure 2: Top 3 predicted probabilities for selected test images.

Most selected examples were classified with high confidence. For example, the shown american football, basketball, football, and tennis examples were predicted with probabilities close to or above 99%.

9 Milestone 2 Updated Pipeline

In Milestone 2, the classical image processing pipeline was:

Read → Preprocess → Segment → Extract Features → Combine Features → Classify → Evaluate

The preprocessing stage resized the images to 256×192 , applied Gaussian blur, and converted images to HSV. A multi-strategy segmentation approach was used, combining HSV thresholding, Otsu thresholding, GrabCut, and a central-crop fallback. The mask was refined using morphological operations and largest connected component selection.

The best Milestone 2 classifier was **Random Forest + LBP**, achieving **89.97%** test accuracy.

10 Milestone 2 Feature Vector

The output feature vector in Milestone 2 had **3,610 dimensions**. It combined colour, shape, gradient, and texture features.

Table 7: Milestone 2 output feature vector.

Feature Block	Dimensions	Description
Masked HSV colour histogram	48	16 bins for each HSV channel
HOG on mask bounding box	1,764	Gradient descriptor from segmented ball region
HOG on central crop	1,764	Mask-independent gradient descriptor
Shape statistics	8	Aspect ratio, solidity, circularity, Hu moments, etc.
Masked LBP texture histogram	26	Texture descriptor for dimples, stitching, and surface
Total	3,610	Combined feature vector

11 Milestone 2 Confusion Matrix and Runtime

Table 8: Milestone 2 Random Forest + LBP confusion matrix.

True / Predicted	american_football	baseball	basketball	football	golf_ball	tennis_ball
american_football	43	6	0	0	0	1
baseball	1	45	0	2	0	1
basketball	7	1	39	1	0	2
football	2	1	1	41	0	0
golf_ball	1	0	0	2	42	0
tennis_ball	0	0	0	0	0	50

The logged Milestone 2 runtime was dominated by feature extraction. Training feature extraction required 187 seconds, and test feature extraction required 36 seconds. The Random Forest fitting time was not separately logged in the Milestone 2 report.

Table 9: Milestone 2 logged runtime.

Operation	Runtime
Training feature extraction	187 seconds
Test feature extraction	36 seconds
Random Forest fitting time	Not separately logged

12 Comparison Between Milestone 2 and Milestone 3

Table 10: Comparison between Milestone 2 and Milestone 3.

Milestone	Method	Test Accuracy
Milestone 2	Random Forest + LBP	89.97%
Milestone 3	Custom CNN V4 from scratch	97.58%
Improvement	–	+7.61 percentage points

The Milestone 3 model improved the test accuracy by:

$$97.58\% - 89.97\% = 7.61\%$$

This improvement shows the advantage of learning image features directly through convolutional layers instead of relying only on manually designed colour, texture, shape, and gradient descriptors.

13 Discussion

The custom CNN performed well because it combines several useful manually configured components. Convolutional layers extract local spatial patterns, residual connections improve training stability, and squeeze-excitation blocks help the model emphasize the most informative channels. The high-resolution input size of 256×256 helped preserve details such as stitching, dimples, seams, panels, and surface texture.

The model had very strong recall for basketball and tennis ball, both achieving 100% recall. The remaining errors were mainly between classes with similar visual cues or difficult image conditions. For example, baseball and american football can both contain white or leather-like regions, while golf ball and tennis ball can both appear as bright circular objects.

14 Conclusion

A fully custom CNN was developed for Milestone 3 using only manually configured deep learning layers. The final model avoided predefined deep learning architectures and pretrained weights. It achieved **97.58%** test accuracy on the unchanged test set, with macro precision of **0.9771**, macro recall of **0.9750**, and macro F1-score of **0.9757**.

Compared with the Milestone 2 Random Forest + LBP model, the custom CNN improved the accuracy by **7.61 percentage points**. This demonstrates that a carefully designed CNN from scratch can outperform the classical image processing pipeline while still respecting the project rule against predefined deep learning models.

A Code Availability

The full implementation is submitted separately as `Milestone3_Code.ipynb`. The submitted notebook includes dataset extraction, dataset auditing, high-resolution data loading, custom CNN V4 construction, training, final evaluation, confusion matrix generation, top-3 probability visualization, and final summary generation.

B Core Architecture Code Excerpt

```
def residual_se_block(x, filters, strides=1, dropout=0.0, weight_decay=8e-5):
    shortcut = x

    x = Conv2D(filters, 3, strides=strides, padding='same', use_bias=False)(x)
    x = BatchNormalization()(x)
    x = Activation('swish')(x)

    x = Conv2D(filters, 3, padding='same', use_bias=False)(x)
    x = BatchNormalization()(x)

    # Manually implemented squeeze-excitation attention
    s = GlobalAveragePooling2D()(x)
    s = Dense(max(filters // 8, 8), activation='relu')(s)
    s = Dense(filters, activation='sigmoid')(s)
    s = Reshape((1, 1, filters))(s)
    x = Multiply()([x, s])

    if shortcut channels or size do not match:
        shortcut = Conv2D(filters, 1, strides=strides, padding='same')(shortcut)
        shortcut = BatchNormalization()(shortcut)

    x = Add()([x, shortcut])
    x = Activation('swish')(x)
    x = Dropout(dropout)(x)
    return x
```